

DSPO: STABLE AND EFFICIENT POLICY OPTIMIZATION FOR AGENTIC SEARCH AND REASONING

Anonymous authors
Anonymous submission

ABSTRACT

LLMs increasingly need to search external knowledge for complex and real-world tasks, but existing agentic-search methods show a clear empirical gap: prompting-only systems leave policy learning implicit, while RL-based systems often hit performance ceilings or collapse during complex interaction. We introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, an RL algorithm that makes two comparison-critical changes to prior objectives: sequence-level optimization aligns updates with terminal trajectory rewards, and dynamic sample filtering keeps each batch informative under sparse binary feedback. Trained purely with RL, our model learns to interleave multi-turn search and reasoning without supervised demonstrations. Across multiple QA benchmarks, DSPO delivers a **34.1%** relative gain over the comparable 7B prior model and, on complex multihop QA such as HotpotQA, a 7B DSPO model exceeds the prior 14B model by nearly **9% relative**, while preserving the stable training behavior that the baselines lack.

1 INTRODUCTION

Large Language Models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Zhao et al., 2023) now achieve strong results on specialized tasks such as math (Shao et al., 2024; Trinh et al., 2024; Yu et al., 2025), coding (Zheng et al., 2023; Yang et al., 2024), and creative writing (Chakrabarty et al., 2024; Marco et al., 2024). Yet these gains are bounded by a measurable deployment constraint: the model’s knowledge remains fixed by its training data. A common way to close this gap is to give the LLM a search interface, turning it into an agent that can query external evidence before answering (Jin et al., 2025b). This tool-calling setting (Schick et al., 2023) requires more than retrieving a document once; the model must learn a multi-step policy for deciding when to reason, when to search, and when enough evidence has been collected. That structure makes agentic search a natural RL problem, where the quality of the learned policy can be evaluated directly through downstream answer accuracy.

Compared with Supervised Fine-Tuning (SFT), which depends on costly static demonstrations and provides limited exploration, RL can improve search behavior through trial-and-error feedback (Chu et al., 2025). Value-free algorithms such as Group Relative Policy Optimization (GRPO) (Shao et al., 2024) are especially attractive because they avoid a separate value model and reduce memory cost. However, the empirical pattern in interactive search differs from more constrained reasoning tasks: GRPO-style training is prone to instability and underperformance (Jin et al., 2025b; Yu et al., 2025; Cui et al., 2025; Liu et al., 2025). Two factors explain this gap. First, as noted by Zheng et al. (2025), the token-level GRPO objective is mismatched with sequence-level terminal rewards, producing high-variance gradients over long trajectories. Second, sparse binary rewards often produce rollout groups in which every sample succeeds or every sample fails; in those cases, the group-relative advantage collapses and the batch contributes little or no learning signal (Yu et al., 2025; Liu et al., 2025).

To close these stability and sample-efficiency gaps, we introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**. DSPO combines two mechanisms whose roles are directly reflected in the experiments. It adopts sequence-level optimization from GSPO (Zheng et al., 2025), so the optimization unit matches the trajectory-level reward, and it adds a dynamic outcome-based filter inspired by DAPO (Yu et al., 2025), so retained rollout groups contain both successful and unsuccessful outcomes. The resulting batches provide effective, non-zero estimates of $\hat{A}_i$, while the update itself avoids token-level ratio noise. In aggregate, this design yields a stable, high-performing algorithm for multi-turn search and reasoning: our 7B model obtains a **34.1% relative improvement** over a leading 7B baseline (Jin et al., 2025b) and exceeds the corresponding 14B model (Jin et al., 2025a) on HotpotQA by **nearly 9% relative** (0.613 vs. 0.563).

In summary, our main contributions are as follows:

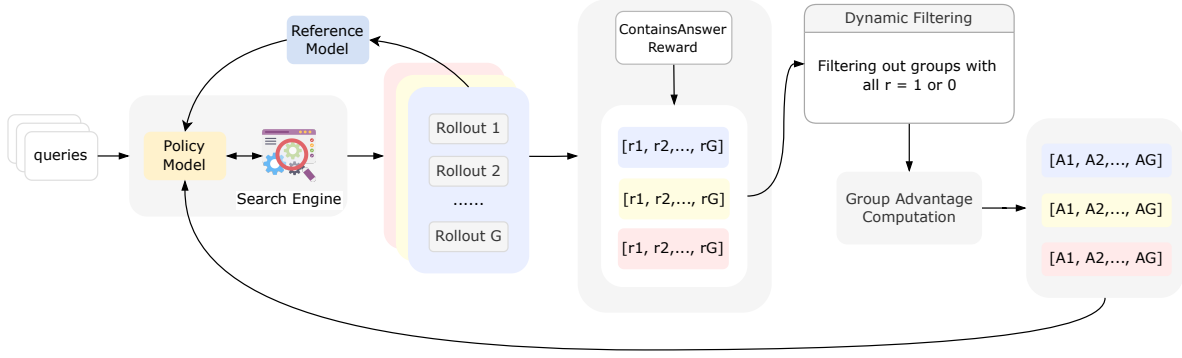


Figure 1: Overview of the DSPO training loop and its comparison-relevant control points. For each query, the policy generates a group of G search trajectories, each scored by a sparse terminal reward. The **dynamic filter** removes homogeneous all-success or all-failure groups and continues sampling until the batch contains informative groups, so every retained batch supports a non-zero advantage estimate. The final update then uses a sequence-level objective, aligning the optimization unit with the trajectory-level reward.

- We propose DSPO, an RL algorithm for agentic search whose two components target the main empirical failure modes of prior training: **sequence-level optimization** stabilizes policy updates, and **dynamic outcome-based filtering** keeps sparse-reward batches informative.
- We quantify DSPO’s gains against strong external baselines and internal ablations. The 7B DSPO model achieves a **34.1% relative improvement** over the comparable 7B baseline and outperforms the 14B counterpart on HotpotQA, with a nearly **9% relative gain** (0.613 vs. 0.563).
- We present empirical evidence that the accuracy margins coincide with stable reward and validation trajectories. Because all search experiments use the same basic BM25 retriever, the comparisons isolate the observed gains to the policy-optimization algorithm rather than retrieval strength.

2 RELATED WORK

2.1 RL FOR LLMs

RL for LLMs has moved from RLHF pipelines that combine PPO with explicit reward models (Ouyang et al., 2022; Christiano et al., 2017; Schulman et al., 2017) toward simpler objectives such as DPO (Rafailov et al., 2023). GRPO (Shao et al., 2024) marks an important value-free step because it derives relative advantages from group statistics instead of a learned value function. The subsequent literature can be read as a set of targeted responses to two quantitative failure patterns: unstable token-level updates (Liu et al., 2025; Cui et al., 2025) and weak learning signal under sparse rewards. GSPO shifts the objective to the sequence level to match the reward unit (Zheng et al., 2025), DAPO improves sparse-reward efficiency with dynamic outcome-based sampling (Yu et al., 2025), and GMPO reduces token-ratio sensitivity through geometric-mean aggregation (Zhao et al., 2025). In our agentic-search experiments, however, these ingredients in isolation still leave either collapse behavior or performance bottlenecks. DSPO therefore combines sequence-level optimization with dynamic filtering in one algorithm, so the empirical comparison tests both stability and sample efficiency in the same autonomous-search setting.

2.2 LLMs WITH AGENTIC RETRIEVAL

To reduce static-knowledge errors, RAG systems attach external retrievers to LLMs so generation can condition on retrieved evidence (Lewis et al., 2020; Gao et al., 2023). Classical RAG typically retrieves documents with a dense retriever and concatenates them into the model input (Karpukhin et al., 2020), which improves access to evidence but fixes much of the decision process outside the model. Agentic retrieval shifts the comparison target: the LLM must choose when and how to search across multiple turns. ReAct couples reasoning with tool use for web-style tasks (Yao et al., 2023), multi-agent systems such as AutoGen coordinate collaborative workflows (Wu et al., 2024), and Agentic Reasoning integrates tools into streamlined LLM reasoning (Wu et al., 2025). Recent RL-integrated methods (Jin et al., 2025b; Chen et al., 2025; Song et al., 2025) train this interleaving behavior directly from feedback, avoiding hand-engineered multi-agent pipelines. The remaining empirical limitation is that open-ended search magnifies instability

and caps performance. DSPO addresses this specific comparison point by making the RL update more stable and the sparse-reward batches more consistently useful.

3 METHODOLOGY

This section defines the comparison setting used throughout the paper. We first formulate agentic search as an RL problem and identify where standard PPO- and GRPO-style objectives lose stability or signal. We then introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, emphasizing how each component changes the quantity being optimized or the quality of the sampled batch. Finally, we present the integrated training algorithm used in the experiments.

3.1 PRELIMINARIES

Policy Gradient Methods for LLMs. RL training for LLMs commonly starts from policy-gradient methods such as PPO (Schulman et al., 2017), a standard alignment baseline. PPO updates a policy π_θ with samples from an old policy $\pi_{\theta_{\text{old}}}$ and constrains the update through a clipped surrogate objective. In the token-averaged form most relevant to our comparison, the objective is:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the token-level importance ratio. The practical cost of this baseline is that PPO needs a separately trained value model to estimate token-level advantages $\hat{A}_t$ via Generalized Advantage Estimation (GAE) (Schulman et al., 2015). That value component increases memory use and introduces another source of variance in long search trajectories.

GRPO (Shao et al., 2024) removes this value-model requirement by sampling a group of G responses $\{y_i\}_{i=1}^G$ for the same prompt x and computing each response advantage from the group’s reward statistics. This makes GRPO a stronger efficiency baseline than PPO, but the optimization unit remains token-level:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{KL}(\pi_\theta || \pi_{\text{ref}}) \right], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$. Although the ratio is token-specific, the advantage assigned to every token $y_{i,t}$ in response y_i is the same sequence-level value:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}, \quad (3)$$

This exposes the key mismatch used in our later ablations: all tokens in y_i receive the shared advantage $\hat{A}_i$, but the clipping and importance weighting are still applied token by token.

Agentic Search as a Markov Decision Process. We model agentic search and reasoning as a discrete-time, finite-horizon Markov Decision Process (MDP), making explicit which quantities are observed, optimized, and evaluated in the experiments.

- **State (s_t):** At turn t , the state s_t contains the full interaction history: the initial question q , previous thoughts and search queries, and all retrieved evidence returned by the environment. This expanding state is important empirically because later answers depend on earlier search choices, creating long-horizon dependencies where token-level RL updates can accumulate noise.
- **Action (a_t):** An action is a full textual segment generated by the policy π_θ , consisting of free-form reasoning followed by a decision. The action terminates either with a `</tool_call>` token, which triggers a search and places results inside `</tool_response>`, or with a `</answer>` token, which ends the trajectory. Thus the learned policy is evaluated not only on *what* it generates but also on *when* it searches, a choice that produces variable trajectory lengths across examples.

- **Policy (π_θ):** The policy π_θ is the underlying LLM, which generates tokens autoregressively conditioned on the current state. Although the policy reads the full history, the optimization target is computed only on model-generated thoughts and actions; retrieved environment content is masked out (Jin et al., 2025b). This isolates the learned behavior being compared across methods: credit assignment over long model-generated sequences with no direct supervision for intermediate steps.
- **Trajectory (τ):** A trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ records the complete reasoning-and-search process for a question, including model actions and environment-returned search results. Since the final metric is determined after the entire trajectory, early search decisions and later answer generation jointly determine reward.
- **Reward ($R(\tau)$):** We use the same sparse terminal reward throughout the comparisons: a trajectory receives $R = 1$ if the final answer contains the ground-truth text and $R = 0$ otherwise:

$$R(\tau) = \begin{cases} 1 & \text{if } a_{\text{gold}} \subseteq a_{\text{pred}}, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

RL with a Search Engine. Following Jin et al. (2025b), we treat the search engine $\mathcal{S}$ as part of the environment rather than as a fixed preprocessing step. The policy LLM π_θ generates trajectories by alternating between reasoning tokens and calls to $\mathcal{S}$. The optimization target is expected terminal reward, regularized by KL divergence from a reference policy π_{ref} so that performance gains do not come from unconstrained policy drift:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x; \mathcal{S})} [R(x, y)] - \beta D_{\text{KL}} [\pi_\theta(y|x; \mathcal{S}) || \pi_{\text{ref}}(y|x; \mathcal{S})]. \quad (5)$$

Here, $y \sim \pi_\theta(\cdot|x; \mathcal{S})$ denotes a full multi-step trajectory produced by both model token generation and search-engine responses. This notation matches the evaluation condition: methods are compared on final answer quality after the complete interaction.

Motivation for DSPO. Search-R1 (Jin et al., 2025b) establishes this RL formulation for agentic search, but conventional PPO or GRPO objectives do not fully match its empirical structure. The reward is sequence-level, whereas the conventional update is token-level, creating the mismatch between optimization unit and reward unit (Zheng et al., 2025) and producing high-variance gradient estimates over long trajectories. At the same time, the binary reward can make an entire rollout group homogeneous, so the normalized advantage becomes uninformative and sample efficiency falls (Yu et al., 2025; Liu et al., 2025). DSPO is designed around these two measurable failure modes: collapse from token-level instability and wasted updates from zero-signal groups.

3.2 DYNAMIC-FILTER SEQUENCE-LEVEL POLICY OPTIMIZATION

DSPO changes both sides of the comparison. First, it performs policy optimization at the sequence level, aligning the update with the same trajectory unit used by the reward. Second, it applies dynamic filtering so that each retained training group contains both positive and negative outcomes, producing a non-zero advantage signal. Figure 1 shows how these two mechanisms are combined in the training loop.

3.2.1 SEQUENCE-LEVEL POLICY OPTIMIZATION FOR ENHANCED STABILITY

Following the sequence-level motivation of GSPO (Zheng et al., 2025), DSPO replaces per-token importance ratios with one trajectory-level ratio for each response. For response y_i , the sequence-level importance ratio $s_i(\theta)$ is the geometric mean of its token-level ratios:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\text{old}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\text{old}}(y_{i,t}|x, y_{i,<t})} \right). \quad (6)$$

The length normalization makes ratios more comparable across short and long trajectories, reducing variance and keeping $s_i(\theta)$ in a range where clipping has a consistent effect across evaluation conditions.

Gradient Analysis. The gradient comparison highlights the stability mechanism tested in our ablations. The unclipped token-level GRPO objective scales each token’s log-probability gradient by its own potentially noisy weight

$r_{i,t}(\theta)$. DSPO instead applies one sequence-level weight $s_i(\theta)$ to the trajectory-level gradient:

$$\nabla_{\theta} J_{\text{GRPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot \sum_{t=1}^{|y_i|} r_{i,t}(\theta) \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) \right] \quad (7)$$

$$\nabla_{\theta} J_{\text{DSPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot s_i(\theta) \cdot \sum_{t=1}^{|y_i|} \nabla_{\theta} \log \pi_{\theta}(y_{i,t} | \dots) \right] \quad (8)$$

By applying a single correction factor to the whole trajectory, DSPO reduces the accumulation of token-level ratio noise that drives instability in the token-level baselines. Dynamic filtering complements this by ensuring that $\hat{A}_i$ is computed from rollout groups with both successes and failures, so the stable sequence-level update is also supplied with an informative sparse-reward signal.

3.2.2 DYNAMIC OUTCOME-BASED FILTERING FOR EFFICIENT LEARNING

The binary reward creates a direct sample-efficiency problem for group-relative estimation. When all G responses in a group are correct ($R = 1$) or all are incorrect ($R = 0$), the normalized advantage $\hat{A}_i$ becomes zero or undefined, so the batch consumes rollouts without producing a useful gradient signal.

DSPO therefore incorporates a dynamic filter inspired by DAPO (Yu et al., 2025). During sampling, it retains only groups with a mixture of successful and unsuccessful outcomes. A group $\{y_i\}_{i=1}^G$ is used for training only if its rewards $\{R_i\}_{i=1}^G$ satisfy:

$$0 < \sum_{i=1}^G R_i < G. \quad (9)$$

This condition guarantees non-zero within-group reward variance for every retained group. As a result, each policy update is computed from groups that can rank successful trajectories against unsuccessful ones, converting sparse terminal feedback into a consistently usable learning signal.

3.3 THE DSPO OBJECTIVE AND TRAINING ALGORITHM

Combining sequence-level ratios with the filtered sample space yields the final DSPO objective. For each valid group from $\mathcal{D}_{\text{filtered}}$, the advantage $\hat{A}_i$ is computed by the same group-relative normalization used in the comparisons:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \delta}, \quad (10)$$

where δ is a small constant for numerical stability. DSPO then updates the policy π_{θ} by maximizing the clipped sequence-level objective:

$$J_{\text{DSPO}}(\theta) = \mathbb{E}_{(x, \{y_i\}) \in \mathcal{D}_{\text{filtered}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right], \quad (11)$$

where s_i is the sequence-level importance ratio defined as the geometric mean of the token-level ratios. During likelihood calculation, we follow Jin et al. (2025b) and mask all tokens retrieved from the search tool. This keeps the measured learning signal focused on the model’s reasoning and action choices rather than on copying environment text. Algorithm 1 gives the full training procedure.

4 EXPERIMENTS

The experiments evaluate DSPO along three quantitative axes: final QA accuracy, reward and validation stability during training, and ablation margins for the two algorithmic components. We compare DSPO against Search-R1 PPO/GRPO baselines and against DSPO variants that remove dynamic filtering or sequence-level optimization, so the reported gains can be attributed to both the full method and its individual design choices.

4.1 EXPERIMENTAL SETUP

Following Search-R1 (Jin et al., 2025b), we use a fixed prompt template for all compared methods so that differences in accuracy are driven by training rather than prompting. As shown in Table 1, the template specifies the action markers `<think>`, `<tool_call>`, and `<answer>` used during search.

Algorithm 1 Dynamic-filter Sequence-level Policy Optimization (DSPO)

```
1: Input: Initial policy  $\pi_{\theta_0}$ , fixed reference policy  $\pi_{\text{ref}}$ , prompt dataset  $\mathcal{D}$ , group size  $G$ , batch size  $B$ , search tool  $\mathcal{R}$ .
2: Initialize policy  $\pi_{\theta} \leftarrow \pi_{\theta_0}$ .
3: for each training step do
4:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ .
5:   Initialize training buffer  $\mathcal{B} \leftarrow \emptyset$ .
6:   while  $|\mathcal{B}| < B$  do
7:     Sample a prompt  $x \sim \mathcal{D}$ .
8:     Generate a group of  $G$  trajectories  $\{y_i\}_{i=1}^G$  using  $\pi_{\theta_{\text{old}}}$  and the search tool  $\mathcal{R}$ .
9:     Compute terminal rewards  $\{R_i\}_{i=1}^G = \{\text{ContainsAnswer}(y_i, y_{\text{gold}})\}_{i=1}^G$ .
10:    if  $0 < \sum_{i=1}^G R_i < G$  then ▷ Dynamic outcome-based filtering
11:      Add  $(x, \{y_i\}_{i=1}^G, \{R_i\}_{i=1}^G)$  to  $\mathcal{B}$ .
12:    end if
13:  end while
14:  for each  $(x, \{y_i\}, \{R_i\})$  in  $\mathcal{B}$  do
15:    Compute advantages  $\{\hat{A}_i\}_{i=1}^G$  via group normalization of  $\{R_i\}$ .
16:    Compute sequence-level importance ratios  $\{s_i(\theta)\}_{i=1}^G$  using Eq. 6.
17:    Compute the DSPO loss for the group using Eq. 11, applying masks to retrieved tokens.
18:  end for
19:  Update policy parameters  $\theta$  by taking a gradient step on the total loss from  $\mathcal{B}$ .
20: end for
```



Figure 2: **Validation performance of DSPO across seven benchmarks during training.** Accuracy increases consistently across the benchmark suite, showing that the reward gains observed during training correspond to improved held-out QA performance rather than only to optimization of the training reward.

Benchmarks and Baselines. For a direct and reproducible comparison, we follow the Search-R1 protocol (Jin et al., 2025b). Training uses a composite dataset built from the training splits of Natural Questions (NQ) (Kwiatkowski et al., 2019) and HotpotQA (Yang et al., 2018). Generalization is then measured on seven QA test sets that span single-hop, multi-hop, and long-tail knowledge settings: NQ, TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), HotpotQA, 2WikiMultiHopQA (Ho et al., 2020), Musique (Trivedi et al., 2022), and Bamboogle (Press et al., 2022).

The comparison suite separates external competitiveness from internal attribution. External baselines are Qwen2.5-7B and 14B Search-R1 models trained with PPO and GRPO (Jin et al., 2025b;a). Internal ablations remove one DSPO component at a time: **DSPO w/o dynamic filter**, equivalent to GSPO (Zheng et al., 2025), tests the value of filtering informative groups, while **DSPO w/o sequence-level opt.**, which reverts to a strong token-level policy, DAPO (Yu et al., 2025), tests the value of sequence-level updates.

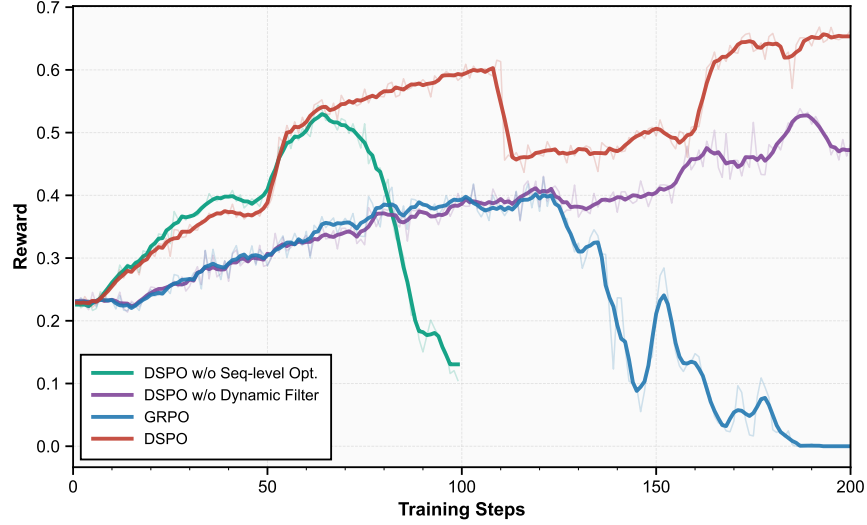


Figure 3: **Training reward dynamics of DSPO and its ablations.** The learning curves compare the full method with variants that remove one source of stability or signal. DSPO (red) improves monotonically, token-level variants (green, blue) collapse after early gains, and the sequence-level variant without dynamic filtering (purple) remains stable but plateaus at a lower reward.

Prompt Template. Answer the given question. Whenever new information is available, first reason inside `<think>` and `</think>`. If that reasoning shows that external knowledge is needed, call the search engine with `<tool_call>` query `</tool_call>`; the environment returns top search results between `<tool_response>` and `</tool_response>`. You may search multiple times. When no further external knowledge is needed, provide the final answer inside `<answer>` and `</answer>` without detailed illustrations. For example, `<answer>` Beijing `</answer>`. **Question:** ...

Table 1: Prompt template held fixed across all search experiments, defining the reasoning, search-call, search-response, and answer markers used by every compared method.

For implementation, all 7B training starts from Qwen2.5-7B-Instruct. The experiments use the VeRL framework (Sheng et al., 2025), with adaptations of the provided Search-R1-like example code and scripts. Because our reward function differs from the originally reported Search-R1 setup, we retrain the PPO and GRPO baselines (Jin et al., 2025b;a) under the same experimental conditions used for DSPO. These retrained results are the primary external comparison, while original Search-R1 EM scores are retained in Table 2 for context.

Implementation and Evaluation. To isolate the algorithmic contribution, every RL search experiment uses the same standard BM25 retriever. Thus, differences in subEM accuracy reflect the learned search-and-reasoning policy rather than stronger retrieval infrastructure. All methods are trained with the same sparse binary reward based on substring Exact Match (subEM) of the final answer, and subEM is also the primary evaluation metric.

4.2 MAIN RESULTS AND ABLATION STUDY

Table 2 places the main benchmark comparison and the ablation study in one view, making the margins attributable to DSPO explicit. The columns compare external Search-R1 PPO/GRPO baselines at 7B and 14B scale with the full 7B DSPO model, while the ablation columns show how much performance is lost when dynamic filtering or sequence-level optimization is removed.

Comparison with Baselines. Table 2 shows that the 7B DSPO agent achieves the strongest average score, 0.531, across the seven QA benchmarks. Relative to the same-sized Search-R1 GRPO baseline, this is a 34.1% improvement, and the average slightly exceeds both larger 14B Search-R1 variants. The dataset-level pattern is also favorable: DSPO is best or second-best on most benchmarks, including a HotpotQA score of 0.613 compared with 0.563 for the 14B

Table 2: Comprehensive subEM comparison across seven QA benchmarks. DSPO is compared with Search-R1 GRPO/PPO baselines at 7B and 14B scale (Jin et al., 2025b;a) and with two ablations that remove dynamic filtering or sequence-level optimization. Parenthesized values are original Search-R1 EM scores; non-parenthesized baseline values are retrained under our adjusted reward for evaluation consistency. Best results are in **bold**; second-best results are underlined.

Dataset	Search-R1				DSPO & Ablations (Ours, 7B)		
	GRPO (7B)	PPO (7B)	GRPO (14B)	PPO (14B)	w/o dyn. filter	w/o seq-lvl opt.	DSPO
NQ	0.423 (0.429)	(0.393)	<u>0.535</u> (0.482)	(0.424)	0.363	0.470	0.580
TriviaQA	0.658 (0.623)	(0.610)	0.760 (0.667)	(0.660)	0.515	0.695	<u>0.754</u>
PopQA	0.395 (0.427)	(0.397)	<u>0.477</u> (0.434)	(0.442)	0.277	0.430	0.498
HotpotQA	0.401 (0.386)	(0.370)	<u>0.563</u> (0.429)	(0.436)	0.330	0.438	0.613
2WikiMultiHopQA	0.357 (0.414)	(0.346)	0.611 (0.424)	(0.379)	0.285	0.398	<u>0.569</u>
Musique	0.122 (0.162)	(0.146)	<u>0.260</u> (0.191)	(0.210)	0.105	0.133	0.270
Bamboogle	0.280 (0.400)	(0.368)	0.504 (0.492)	(0.480)	0.288	0.280	<u>0.432</u>
Average	0.377 (0.396)	(0.385)	<u>0.530</u> (0.446)	(0.433)	0.313	0.406	0.531

GRPO baseline. These margins indicate that the gains are not simply a scale effect, but are tied to the more stable and informative policy-optimization procedure.

Analysis of Ablations. The ablation columns quantify the contribution of each component. Removing the dynamic filter reduces the average score to 0.313, a large drop from the full DSPO average of 0.531, showing that sequence-level updates alone are not enough when many sparse-reward groups lack contrast. Removing sequence-level optimization yields an average of 0.406: this token-level variant benefits from filtering but remains far below the full method and, as shown in the training-dynamics analysis, is vulnerable to collapse. The comparison therefore supports the intended division of labor: sequence-level optimization supplies update stability, while dynamic filtering supplies reliable within-group reward contrast.

The quantitative gains are accompanied by qualitative behaviors consistent with the metrics: DSPO learns to recognize irrelevant results, reformulate queries, and perform multi-turn verification (see Appendix A for detailed trajectories). These behaviors emerge from pure RL training and help explain the practical significance of the benchmark margins.

4.3 ANALYSIS OF TRAINING DYNAMICS

To validate that the accuracy improvements arise from stable learning rather than isolated final checkpoints, we compare reward trajectories for DSPO, its ablations, and key baselines. Figure 3 makes the stability pattern visible across methods and connects the final benchmark margins to training behavior.

DSPO (red) shows a smooth, monotonic reward increase and converges to the highest level among the compared methods. The two token-level methods—DSPO w/o Seq-level Opt. (green) and vanilla GRPO (blue)—initially improve but then collapse, matching the expected instability from high-variance token-level ratios. The sequence-level variant without dynamic filtering (purple) remains stable, but its reward plateaus below the full model because many updates lack an informative sparse-reward contrast. Together, these curves mirror the ablation table: each component addresses a different empirical failure mode, and the full method is needed for both stability and high reward.

We also track validation accuracy throughout training to test whether reward improvements transfer to held-out benchmarks. Figure 2 shows that DSPO’s accuracy rises consistently on NQ, HotpotQA, and the broader benchmark set, closely following the reward trajectory. This agreement between reward and validation metrics supports the interpretation that DSPO learns a generalizable search-and-reasoning policy rather than exploiting the training reward.

4.4 SCALABILITY AND GENERALIZATION ANALYSIS

We further test whether the empirical advantages persist beyond the primary 7B search setting by evaluating model scalability and transfer to mathematical reasoning.

Scalability to Larger Models. To test scaling, we train Qwen2.5-14B-Instruct with DSPO and compare it with the 14B Instruct and GRPO baselines in Table 3. The DSPO-trained 14B model reaches an average accuracy of 60.6%, compared with 53.0% for the GRPO-14B baseline, a **14.3%** relative gain. The improvements are consistent across all listed datasets, with especially large margins on Musique (+38.8%), HotpotQA (+18.1%), and NQ (+17.6%). This pattern shows that DSPO continues to benefit from increased capacity rather than only improving the smaller 7B setting.

Table 3: Scalability analysis on Qwen2.5-14B-Instruct. DSPO-14B is compared with the 14B Instruct model, the 14B GRPO baseline, and the 7B DSPO model; the gain column reports the relative improvement of DSPO-14B over GRPO-14B. Best results are in **bold**.

Dataset	Instruct (14B)	GRPO (14B)	DSPO (7B)	DSPO (14B)	Gain
NQ	0.345	0.535	0.580	0.629	+17.6%
HotpotQA	0.407	0.563	0.613	0.665	+18.1%
2WikiMQA	0.332	0.611	0.569	0.699	+14.4%
Bamboogle	0.328	0.504	0.432	0.544	+7.9%
PopQA	0.364	0.477	0.498	0.545	+14.3%
TriviaQA	0.643	0.760	0.754	0.802	+5.5%
Musique	0.151	0.260	0.270	0.361	+38.8%
Average	0.367	0.530	0.531	0.606	+14.3%

Generalization to Mathematical Reasoning. We next evaluate whether the same optimization principles transfer outside agentic search. Table 4 compares DSPO with GRPO on single-turn mathematical reasoning tasks using Qwen2.5-Math-7B on Math500 and Qwen3-4B on Olympiad-Bench. DSPO improves over GRPO in both settings, with gains of +2.6% and +2.7%, respectively. Although smaller than the search-domain margins, the consistent positive differences suggest that sequence-level stable updates and informative sampling can also benefit general reasoning tasks.

Table 4: Generalization to mathematical reasoning. DSPO is compared with GRPO at the same model size, benchmark, and training-step budget; the gain column reports the relative improvement of DSPO over GRPO. Best results are in **bold**.

Model	Benchmark	Steps	GRPO	DSPO	Gain
Qwen2.5-Math-7B	Math500	200	0.772	0.798	+2.6%
Qwen3-4B	Olympiad-Bench	100	0.728	0.755	+2.7%

5 CONCLUSION

This work addressed two empirically visible obstacles in RL for autonomous LLM search agents: unstable token-level policy updates and low-signal sparse-reward batches. Dynamic-filter Sequence-level Policy Optimization (DSPO) targets both obstacles by matching the update unit to the sequence-level reward and by retaining only rollout groups with outcome contrast. Across seven QA benchmarks, the 7B DSPO model reaches an average score of 0.531, a 34.1% relative gain over the comparable 7B Search-R1 GRPO baseline, while also matching or exceeding larger 14B Search-R1 baselines on average.

The broader empirical picture is consistent across final accuracy, ablations, training curves, scaling, and math transfer. Removing dynamic filtering drops the average to 0.313, removing sequence-level optimization drops it to 0.406, and the training dynamics show the corresponding failure modes: token-level variants collapse, whereas sequence-level training without filtering plateaus. At 14B scale, DSPO improves over GRPO-14B by 14.3% on average, and on Math500 and Olympiad-Bench it also outperforms same-size GRPO baselines. These results show that DSPO provides a practical route to training capable search agents from environmental feedback alone, without supervised expert trajectories. Future work can build on this stable optimization foundation by adding stronger retrievers, extending to multi-tool settings, and testing whether the same principles deliver larger gains in other sparse-reward domains such as code generation.

REFERENCES

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Tuhin Chakrabarty, Vishakh Padmakumar, Faeze Brahman, and Smaranda Muresan. Creativity support in the age of large language models: An empirical study involving professional writers. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 132–155, 2024.
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Guillermo Marco, Julio Gonzalo, Ramón del Castillo, and María Teresa Mateo Girona. Pron vs prompt: Can large language models already challenge a world-class fiction author at creative text writing? *arXiv preprint arXiv:2407.01119*, 2024.

- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilinfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

-
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5673–5684, 2023.

A APPENDIX

Table A.1 provides representative trajectories that contextualize the quantitative improvements. The examples show behaviors that are useful for higher subEM accuracy under multi-turn search: reformulating queries after irrelevant retrievals, verifying information through follow-up searches, and extracting final answers from retrieved evidence.

Table A.1: Representative trajectories illustrating behaviors that accompany DSPO’s benchmark gains. The examples show (a) recognition of irrelevant results, (b) query reformulation after failed retrieval, and (c) verification before final answering.

Example 1: Presidential Succession (<i>Query Refinement after Failure</i>)	
Question	Who is next in line if something happens to the president?
Gold	vice president
Turn 1	search("who is next in line if something happens to the president") → Retrieved: Documents about novels "Something Happened" and "Mosquitoes Buzz" Agent: "The queries were not directly about the line of succession... The relevant information I need is not present in the search results. Let's try searching again with a more specific query."
Turn 2	search("line of succession for the president") → Retrieved: Philippine and US presidential succession information Agent: "Based on the search results... the next person in line would be the Vice President."
Answer	✓ The Vice President
Example 2: University Identification (<i>Information Verification</i>)	
Question	Which American University is situated in Ithaca, New York?
Gold	Cornell University
Turn 1	search(["American University in Ithaca", "University in Ithaca NY"]) → Retrieved: "Ithaca is home to Cornell University, an Ivy League school..." Agent: "Ithaca, New York is home to Cornell University, which is an Ivy League school. However... it is highly likely that the university in question is Cornell. Let me confirm this."
Turn 2	search(["Cornell University location", "Cornell University New York"]) → Retrieved: Cornell history confirming establishment in Ithaca Agent: "Cornell University is indeed located in Ithaca, New York."
Answer	✓ Cornell University
Example 3: Biographical Information (<i>Direct Extraction</i>)	
Question	What is John Andrew Martin’s occupation?
Gold	lawyer, attorney, politician
Turn 1	search(["John Andrew Martin occupation", "John Andrew Martin job"]) → Retrieved: "American journalist, attorney, soldier, and politician..." Agent: "John Andrew Martin was an American <u>journalist, attorney, soldier, and politician</u>. He represented Colorado in the U.S. House of Representatives."
Answer	✓ Journalist, attorney, soldier, and politician